

Multi-Species Weed Recognition: 9-Class DeepWeeds Classification

A Technical Analysis of the Python Implementation and Jupyter Notebook

This report documents the computational pipeline, mathematical formulation, model training procedure, evaluation design, reproducibility mechanisms, and quality-control architecture implemented by the supplied source code.

Contents

1	Abstract	2
2	Introduction	2
3	Project Overview	2
4	Source Inventory and Execution Model	3
5	Dataset Acquisition and Validation	3
6	Label Resolution and Metadata Construction	4
7	Leakage-Aware Split Protocol	5
8	Data Transforms and Dataset Abstraction	5
9	Model Architectures	6
9.1	8.1 EfficientNet-B0	6
9.2	8.2 MobileNetV3-Large	6
9.3	8.3 ViT-Tiny	6
10	Class Balancing and Mathematical Formulation	7
11	Optimization and Learning Procedure	8
12	Training Loop and Computational Mechanics	8
13	Pilot Selection and Final Experiment Isolation	9
14	Held-Out Evaluation and Error Analysis	10
15	Reproducibility Engineering	10
16	Quality Control and Artifact Audit	11
17	Result Packaging and Artifact Discipline	11
18	Computational Considerations	12
19	Limitations and Technical Considerations	12
20	Overall Technical Assessment	13
21	Conclusion	13
	Appendix A. Code-to-Report Traceability	14
	Appendix B. Default Experimental Configuration	14
	References	14

1 Abstract

The supplied project implements a research-oriented image-classification pipeline for the nine-class DeepWeeds problem, covering eight invasive weed species and a negative/background class. The implementation compares three pretrained architectures - EfficientNet-B0, MobileNetV3-Large, and ViT-Tiny - under a common classification task while allowing architecture-specific input normalization and augmentation details. Beyond model training, the code contains a substantial data-engineering and experimental-control layer: dataset acquisition from multiple sources, archive and image-count validation, official-label resolution, conservative capture-based grouping, deterministic stratified group splitting, training-only class weighting, two-stage transfer learning, checkpoint selection by validation accuracy, held-out test evaluation, artifact generation, runtime instrumentation, and fourteen executable quality-control passes.

The key scientific property of the implementation is that it explicitly separates data preparation, model selection, and final testing. A deterministic five-fold-derived geometry assigns fold 0 to testing, fold 1 to validation, and folds 2-4 to training, while enforcing group disjointness and class completeness. The code explicitly states that this is a leakage-aware reconstruction rather than a byte-for-byte reproduction of the published DeepWeeds fold assignments. The training procedure is likewise structured to avoid test-set exposure: two candidate learning-rate configurations are piloted per architecture, the preferred configuration is selected from validation top-1 accuracy, and the best validation checkpoint is restored before the held-out test pass.

This report explains the project at both the conceptual and implementation levels. It does not reproduce benchmark values, plots, screenshots, or experimental conclusions, because those depend on executing the pipeline against the real dataset and assigned runtime hardware.

2 Introduction

The project addresses supervised visual classification in an agricultural imaging setting. Its computational objective is to map each input image to one of nine fixed classes. The source identifies the task as multi-species weed recognition and includes a negative/background category. The practical motivation is not only to train a classifier, but to construct an experiment that can be run in a constrained Google Colab environment without abandoning basic scientific safeguards.

The implementation is notable because the machine-learning model is only one layer of the system. The notebook also handles acquisition, caching, integrity verification, metadata normalization, grouping, split construction, data loading, training-state management, evaluation, reproducibility artifacts, and automated quality control. As a result, the source is better understood as an end-to-end experimental pipeline rather than as an isolated training script.

The scope of this report is the supplied implementation itself. When a design rationale is explicitly documented in the source, it is stated as such. When the rationale is inferred from the code structure, the report uses cautious language. No biological explanation of observed model errors is inferred beyond what the implementation can actually support.

3 Project Overview

At a conceptual level, the pipeline can be decomposed into nine stages.

1. Environment preparation and reproducibility configuration.
2. DeepWeeds acquisition, caching, extraction, and integrity validation.
3. Official-label resolution and construction of a metadata table.

4. Leakage-aware grouped partitioning into training, validation, and test subsets.
5. Model-specific image transformation and dataset construction.
6. Pilot selection between two learning-rate configurations per architecture.
7. Final two-stage transfer learning with validation-based checkpoint selection.
8. Held-out test inference followed by scalar and per-class evaluation.
9. Artifact audit, runtime reporting, and reproducible packaging.

The implementation deliberately keeps the test set outside the model-selection loop. The validation set is used for both pilot configuration selection and epoch-level checkpoint selection, while the test set is only opened after a final configuration and checkpoint have been fixed.

The source defines the default execution mode as `colab_balanced`. Under that profile, the pilot uses at most 6,000 training examples, whereas final training is allowed up to 120,000 examples. Because the supplied DeepWeeds release is stated to contain 17,509 images, the 120,000 cap is not active for this dataset; in the default configuration the final stage therefore uses the full available training split. The `full` compute profile removes the cap explicitly and increases the pilot ceiling to 10,000 examples.

4 Source Inventory and Execution Model

The standalone Python source contains 78 named functions and one dataset class. The notebook organizes the same computational system into 39 cells, mixing executable code and Markdown documentation. The Markdown is scientifically meaningful: it states the intended protocol, clarifies where the implementation differs from the published DeepWeeds fold files, and documents the reason for grouping images by acquisition metadata rather than by arbitrary image-level randomization.

The source has several layers of abstraction. At the bottom is filesystem and network handling. Above that is metadata construction. The next layer is split generation and dataset transformation. The model layer then defines architectures, parameter freezing, optimization, metric accumulation, checkpointing, and experiment orchestration. The final layer handles diagnostics, quality control, and packaging.

A few implementation details matter for understanding execution order. The notebook creates a general-purpose `train_transform` and `eval_transform`, but the actual training experiments call `build_model_transforms(model_name)` inside `train_experiment`. Therefore, the final experiments use the architecture-specific normalization profile rather than the earlier generic transform objects. Likewise, the global train/validation/test loaders are created for pipeline checks and dataflow validation, whereas each experiment creates its own train and validation loaders so that model-specific transforms and worker lifecycles can be controlled independently.

5 Dataset Acquisition and Validation

The acquisition subsystem is unusually extensive for a classification notebook because it treats data availability and data integrity as explicit experimental prerequisites.

The main entry point is `prepare_dataset_root()`. In automatic mode it first searches Google Drive for a validated archive or a complete extracted DeepWeeds directory. If a usable local copy is already present, it reuses it. Otherwise, the acquisition routine attempts an official Google Drive image route, then a public Kaggle mirror, and finally a Zenodo download. The

final Zenodo route is additionally checked against the size and MD5 checksum returned by the Zenodo API, followed by ZIP-level validation.

The archive validator performs three logically distinct checks. First, the path must represent a supported archive and `zipfile.is_zipfile()` must accept it. Second, the archive must contain exactly the expected 17,509 image files. Third, `ZipFile.testzip()` is used to detect CRC failures. The validation result is memoized by absolute path, file size, and modification time so repeated discovery operations do not rescan a large archive unnecessarily.

The extraction routine also contains a path-safety check. Before extracting a member, the destination path is resolved and compared with the resolved extraction root using `os.path.commonpath`. An archive member that would escape the destination directory raises an exception. This is a concrete security property of the implementation rather than a generic assumption about archive extraction.

Incomplete download artifacts are handled separately. Temporary `.part` files are never silently accepted as final data. A stale partial download is removed unless it independently passes the same strict archive validation, in which case it can be promoted to the final archive. File copies are similarly staged through temporary `.part` paths and atomically renamed into place.

The labels file is downloaded separately from the official DeepWeeds repository and cached locally and, when possible, in Google Drive. The acquisition design therefore separates the large image payload from the relatively small label metadata. This allows later sessions to reuse the expensive binary archive without repeatedly fetching it.

6 Label Resolution and Metadata Construction

The classifier never infers the target class from visual appearance or directory names. The source reads the official `labels.csv` and expects `Filename`, `Label`, and `Species` fields, while also providing normalized-column fallback matching when exact names differ.

Species strings are normalized by stripping surrounding whitespace, converting to lowercase, and collapsing internal whitespace. The resulting values are mapped to stable internal codes:

Code	Class name
DW_CHAP	Chinee apple
DW_LANT	Lantana
DW_PARK	Parkinsonia
DW_PART	Parthenium
DW_PRAC	Prickly acacia
DW_RUBV	Rubber vine
DW_SIWE	Siam weed
DW_SNWE	Snake weed
DW_NEG	Negative

The code also defines the expected numeric labels 0 through 8 and checks that the numeric `Label` agrees with the mapped species code. A mismatch is treated as a fatal metadata error. This is important because it prevents silent disagreement between two label representations.

Image paths are resolved through an in-memory basename-to-path dictionary constructed from the discovered images. Ambiguous duplicate basenames are removed from the fast dictionary rather than guessed. Only unresolved rows use a slower fallback search over a small set of likely roots. This design reduces per-row filesystem overhead while retaining compatibility with nonstandard extraction layouts.

The resulting metadata table contains, among other fields, `path`, `eppo`, `source_image_id`, `class_index`, `class_name`, `trial_id`, `box_id`, and file size. Several columns expected by an earlier schema (`plant_id`, `growth_condition`, bounding-box coordinates) are retained as empty placeholders. This is an explicit compatibility layer, not an assertion that DeepWeeds provides plant bounding boxes in this pipeline.

A particularly important distinction is that the label-resolution fast path does not decode every image. Width and height are left unknown, `status` is marked `not_scanned`, and the source prints that the full image integrity scan was not performed. The implementation therefore avoids manufacturing image-quality statistics it has not actually measured.

7 Leakage-Aware Split Protocol

The split strategy is designed around the filename convention documented in the notebook: identifiers follow the form `YYYYMMDD-HHMMSS-ID`. The strongest grouping candidate used by the implementation is capture date plus instrument identifier. For example, the grouping concept can be written as

$$G_i = \text{date}(x_i) \parallel \text{camera}(x_i), \quad (1)$$

where x_i is the source image identifier and $\parallel$ denotes concatenation into a group key.

The grouping helper produces three candidate levels, ordered from stronger semantic grouping to more conservative fallback: `box_id`, exact capture timestamp, and the unique source image identifier. The first candidate is accepted only when each class occurs in at least five distinct groups, because a class-complete five-fold split requires at least one group per fold. A deterministic sweep over 50 trial seeds is then used to find a `StratifiedGroupKFold` assignment in which every class appears in every fold.

The selected folds are converted into the experiment geometry

$$\mathcal{D}_{\text{test}} = F_0, \quad \mathcal{D}_{\text{val}} = F_1, \quad \mathcal{D}_{\text{train}} = F_2 \cup F_3 \cup F_4, \quad (2)$$

which gives a nominal 20/20/60 partition by fold count, while actual image counts can differ slightly because groups are indivisible.

The code then checks class completeness independently for train, validation, and test. It also forms every pair of split-group sets and requires empty intersections. Thus, when the grouping constraint is satisfied, no group key appears in more than one partition.

The scientific interpretation must be precise. This is a leakage-aware reconstruction based on the available DeepWeeds identifiers; it is not a claim that the exact row-level assignment equals the repository’s published five-fold CSV assignment. The notebook explicitly records this distinction in `split_config.json`, and it saves the complete split metadata plus a SHA-256 checksum.

8 Data Transforms and Dataset Abstraction

All architectures consume 224 by 224 RGB tensors. Training uses mild stochastic augmentation: `RandomResizedCrop` with scale 0.75-1.0 and aspect ratio 0.9-1.1, horizontal flipping with probability 0.5, modest `ColorJitter`, and a small affine perturbation consisting of rotation, translation, and scale variation. Validation and test data use deterministic resize-then-center-crop preprocessing.

The architecture-specific transform builder is important because the CNNs and ViT-Tiny are normalized differently. EfficientNet-B0 and MobileNetV3-Large use the standard ImageNet mean and standard deviation,

$$\mu_{\text{CNN}} = (0.485, 0.456, 0.406), \quad \sigma_{\text{CNN}} = (0.229, 0.224, 0.225), \quad (3)$$

while the ViT configuration uses

$$\mu_{\text{ViT}} = (0.5, 0.5, 0.5), \quad \sigma_{\text{ViT}} = (0.5, 0.5, 0.5). \quad (4)$$

The code also uses bilinear interpolation for the CNNs and bicubic interpolation for the ViT transform profile. Using architecture-aware preprocessing is preferable to forcing all pretrained models through one normalization convention, because pretrained weights were learned under different preprocessing assumptions.

`OPPDPlantDataset` is a compact PyTorch `Dataset`. It converts the pandas columns for paths and integer labels into NumPy arrays once, avoiding repeated pandas indexing from worker processes. Each item opens the image, converts it to RGB, applies the supplied transform, and returns the tensor, integer label, and path. The class name is historically named `OPPDPlantDataset`, but in this code it is simply the dataset adapter used for DeepWeeds metadata.

The `stratified_cap` helper is used only when a sample cap is active. It computes class quotas proportional to the source class distribution, floors the quotas, enforces at least one example per class, and then removes excess allocation before sampling without replacement. The resulting subset is shuffled with the same seed. This means the pilot subset remains approximately class-proportional across architectures.

9 Model Architectures

The project compares two convolutional networks and one transformer under the same nine-class objective.

9.1 8.1 EfficientNet-B0

The code calls `torchvision.models.efficientnet_b0` with default pretrained weights. The final linear layer in `model.classifier` is replaced so that its output dimensionality equals nine. During classifier-only training, only the classifier is trainable. During fine-tuning, the final two blocks of the EfficientNet feature stack are unfrozen.

9.2 8.2 MobileNetV3-Large

MobileNetV3-Large is constructed through `torchvision` with default pretrained weights. Its classifier is replaced by a nine-output linear layer. The fine-tuning stage exposes the final three feature blocks in addition to the classifier.

9.3 8.3 ViT-Tiny

The transformer baseline is instantiated through `timm.create_model` with the pretrained model identifier `vit_tiny_patch16_224`, nine output classes, and an image size of 224. In the head stage, the classification head and, when present, `fc_norm` are trainable. During fine-tuning, the last two transformer blocks plus the final normalization layer are exposed to optimization.

The unfreezing policy is therefore architecture-specific but intentionally shallow. Instead of updating all pretrained parameters, the code updates the classifier first and then exposes only a small portion of the backbone. This limits the number of newly optimized degrees of freedom and reduces the risk and computational cost associated with full-network fine-tuning.

10 Class Balancing and Mathematical Formulation

The loss function is weighted cross-entropy. For a training set with class counts $n_1, \dots, n_C$, total sample count N , and $C = 9$ classes, the unnormalized class weight used by the implementation is

$$\tilde{w}_{-c} = \frac{N}{C n_{-c}}. \quad (5)$$

The weights are then normalized by their arithmetic mean,

$$w_{-c} = \frac{\tilde{w}_{-c}}{\frac{1}{C} \sum_{j=1}^C \tilde{w}_{-j}}. \quad (6)$$

The source computes these weights from `train_df` only, after any final-training cap has been applied. Validation and test labels therefore have no role in the class-weight calculation.

For logits $z_{-i} \in \mathbb{R}^C$, the model converts logits to class probabilities with the softmax function

$$p_{-ic} = \frac{\exp(z_{-ic})}{\sum_{j=1}^C \exp(z_{-ij})}. \quad (7)$$

The target class is y_{-i} . Conceptually, the weighted categorical cross-entropy is

$$\mathcal{L}(\theta) = - \sum_{-i=1}^N w_{-y_{-i}} \log p_{-i, y_{-i}}, \quad (8)$$

with the exact batch reduction governed by PyTorch’s `CrossEntropyLoss`. Because the implementation passes class weights directly to that loss, PyTorch’s weighted-mean reduction uses the sum of selected target weights as the normalization factor for the batch. The class-weight formula itself is therefore the mechanism for reweighting class contribution, while the framework controls the final scalar reduction.

The top- k accuracy calculation is based directly on the ranking of logits. For sample i and class label y_{-i} , top- k correctness is

$$\mathbf{1}\{y_{-i} \in \text{TopK}(z_{-i}, k)\}. \quad (9)$$

The implementation computes top-1 and top-5 correctness from one `topk` operation per batch. Since the task has nine classes, top-5 remains a meaningful complementary metric while top-1 is used for model-selection decisions.

11 Optimization and Learning Procedure

Each experiment has two stages. The first stage trains the classification head while the pretrained backbone is frozen. The second stage unfreezes a small architecture-specific subset of the backbone and continues fine-tuning.

The optimizer is AdamW. In the head-only stage all trainable parameters share the configured head learning rate. In the fine-tuning stage the code constructs two parameter groups: backbone parameters receive the lower backbone learning rate, while classifier/head parameters retain the larger head learning rate. This is a standard discriminative-learning-rate pattern, but the reportable fact here is specifically that the code implements it through named-parameter prefix matching and `requires_grad` filtering.

The source uses cosine annealing separately for each training stage:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T}\right), \quad (10)$$

where T is the number of epochs assigned to the current stage. The code relies on PyTorch’s `CosineAnnealingLR`, with one scheduler instantiated for the head stage and a new scheduler instantiated for fine-tuning.

Validation top-1 accuracy is the checkpoint criterion. A new checkpoint is written only when

$$a_t > a_{\text{best}} + \delta, \quad \delta = 10^{-4}, \quad (11)$$

where a_t is validation top-1 accuracy. During fine-tuning, a patience counter increments when this condition is not met; after two consecutive non-improving epochs, training stops early. The head stage does not trigger early stopping. Importantly, the checkpoint is updated whenever the global best validation accuracy improves, regardless of which stage produced it, and that single best checkpoint is restored at the end.

The default hyperparameter candidates are:

Model	Config.	Head LR	Fine-tune LR
EfficientNet-B0	lr1e-3	10^{-3}	3×10^{-4}
EfficientNet-B0	lr5e-4	5×10^{-4}	10^{-4}
MobileNetV3-Large	lr1e-3	10^{-3}	3×10^{-4}
MobileNetV3-Large	lr5e-4	5×10^{-4}	10^{-4}
ViT-Tiny	lr3e-4	3×10^{-4}	10^{-4}
ViT-Tiny	lr1e-4	10^{-4}	5×10^{-5}

EfficientNet-B0 and MobileNetV3-Large use weight decay 10^{-4} , while ViT-Tiny uses 10^{-2} . Two candidates are piloted per architecture, giving six pilot jobs in total when pilot execution is enabled. The pilot uses one head epoch and one fine-tuning epoch because `PILOT_EPOCHS=2`. The selected candidate is then used for the final training run with one head epoch and up to six fine-tuning epochs.

12 Training Loop and Computational Mechanics

`run_epoch` centralizes the actual batch computation. It switches the model between train and evaluation modes, creates device-resident accumulators for loss and accuracy counts, transfers

images and labels with optional non-blocking copies, performs mixed-precision autocasting when a CUDA device is available, and executes optimizer updates only in training mode.

The implementation supports automatic mixed precision (AMP). On supported CUDA hardware it prefers bfloat16 when `torch.cuda.is_bf16_supported()` returns true; otherwise it uses float16. A gradient scaler is used only for float16 training. On CPU, the code falls back to float32 and disables AMP.

A small but technically important detail is the treatment of BatchNorm. When the network is in training mode, `_set_train_mode` identifies frozen BatchNorm modules and places them back into evaluation mode when none of their parameters are trainable. This prevents frozen convolutional backbones from changing running statistics solely because they are traversed during classifier-only training.

The metric accumulation is intentionally device-oriented. Per batch, the code accumulates the detached scalar loss multiplied by batch size and accumulates integer top-1/top-5 counts on the device. The final loss and accuracies are transferred to Python only once after the epoch. This reduces repeated host-device synchronization.

The DataLoader configuration also reflects the target environment. The number of workers is capped at four. Pinning is enabled when CUDA is available, persistent workers are used when there is at least one worker, and a prefetch factor of two is configured. A seeded `torch.Generator` controls batch shuffling, while `seed_worker` propagates worker-specific seeds into NumPy and Python’s standard random module. Worker processes are explicitly shut down at experiment boundaries to avoid retaining resources across multiple model runs.

The source additionally keeps completed final models on CPU between evaluation stages. This is a memory-management decision: three final networks need not simultaneously occupy accelerator memory simply because they are stored in notebook state.

13 Pilot Selection and Final Experiment Isolation

Pilot experiments are run independently for each architecture/configuration pair against the same validation frame. Each pilot calls `train_experiment`, but with the pilot training subset rather than the full training frame. The best pilot configuration is the row with the largest `best_val_top1`; ties are resolved by preferring the earlier best epoch.

This selection mechanism is deliberately narrow. It does not use Macro F1, weighted F1, training loss, test accuracy, or runtime as the formal model-selection criterion. That makes the experiment protocol simple and auditable: the hyperparameter decision is explicitly tied to validation top-1 accuracy.

Once a configuration is selected, it is written to `selected_configs.json`. Final training then starts a fresh pretrained model for each architecture. This is important because the final model does not continue from a pilot checkpoint; instead, the pilot is treated as a configuration search stage and the final model is retrained from the pretrained initialization on the full final training split.

The source also re-seeds at the start of each `train_experiment`. Thus the random state is reset for each architecture/configuration run, making the normal single-seed experiment reproducible at the experiment-design level. The code does not claim bitwise identity by default because strict deterministic algorithms are disabled unless explicitly requested.

14 Held-Out Evaluation and Error Analysis

After final training, each retained model is moved to the active device and evaluated on the test frame with the model-specific deterministic evaluation transform. Inference is wrapped in `torch.inference_mode()` so no autograd graph is constructed.

For each test image, the code stores the ground-truth class, predicted class, full nine-dimensional probability vector, predicted probability, source path, and correctness flag. This makes the evaluation artifact richer than a single scalar score and supports later inspection without re-running inference.

The scalar metrics computed for each model are accuracy, macro precision, macro recall, macro F1, and weighted F1. Let P_{-c} , R_{-c} , and $F_{-1,c}$ denote per-class precision, recall, and F1. Macro F1 is

$$F_{-1,\text{macro}} = \frac{1}{C} \sum_{-c} 1^C F_{-1,c}, \quad (12)$$

where every class receives equal weight. Weighted F1 instead weights class-specific F1 values by the number of true examples, so its interpretation is dominated more strongly by frequent classes.

The confusion matrix is computed with all nine labels in a fixed order. The error-analysis layer identifies low-recall classes, saves the highest-confidence incorrect predictions, and extracts the most frequent off-diagonal confusion pairs for the best model by test top-1 accuracy. The code explicitly avoids assigning biological causes to these errors because no causal biological analysis is performed.

Although the notebook also contains cells that generate training curves, confusion-matrix figures, and an image montage of selected errors, those artifacts are intentionally not part of this report. The present report is limited to the computational and methodological explanation requested for the academic portfolio.

15 Reproducibility Engineering

Reproducibility is treated as a system property rather than a single random seed. The implementation sets Python’s random seed, NumPy’s random seed, `PYTHONHASHSEED`, and PyTorch seeds, and also seeds CUDA when available. The `STRICT_DETERMINISM` flag controls whether PyTorch uses deterministic algorithm settings. The default is false because the notebook explicitly favors faster cuDNN kernel selection for fixed-size inputs.

The split itself is made reproducible through an explicit seed sweep whose trial sequence is completely determined by the base seed. The final split metadata are saved to CSV and hashed with SHA-256. The selected grouping strategy, selected split seed, fold geometry, and strict-grouping flag are stored in `split_config.json`.

An experiment manifest records the image size, batch size, device, GPU name, seed, determinism setting, AMP state, worker count, compute profile, epoch settings, model names, selected configurations, group count, split summary, validation report, and final-model count. A separate environment manifest records Python, platform, PyTorch, torchvision, timm, NumPy, pandas, scikit-learn, CUDA runtime, and cuDNN versions.

This is stronger than recording a random seed alone because many experiment properties are environment- and data-dependent. A future rerun can inspect the manifest and split checksum before interpreting differences in model behavior.

16 Quality Control and Artifact Audit

The notebook contains fourteen executable QC passes followed by a consolidated validation table. The checks are:

1. required runtime object and result-tree structure;
2. dependency surface and version availability;
3. function and variable dependency availability;
4. architecture-specific transform profiles;
5. runtime dataflow availability;
6. tensor shape and label dtype;
7. scientific split isolation and group leakage;
8. metric-column presence and finiteness;
9. runtime instrumentation fields;
10. GPU/VRAM hygiene for retained final models;
11. reproducibility configuration;
12. numerical sanity for class weights and top- k logic;
13. discriminative optimizer-group behavior; and
14. artifact completeness.

The optimizer-group test is particularly concrete. It constructs an EfficientNet-like dummy network, applies the real `freeze_for_stage` and `build_optimizer` functions, and asserts that both backbone and classifier parameters are trainable in fine-tuning and that the optimizer exposes the expected two learning rates. This tests a nontrivial piece of experiment logic without requiring a full training run.

The quality-control layer is therefore more than a passive report. It actively asserts properties of the dataflow, split, tensors, training configuration, metric generation, and artifacts. Failures raise exceptions, and the final validation CSV/JSON captures the status and details of every pass that ran.

17 Result Packaging and Artifact Discipline

The packaging routine is designed as a final export center. It captures open Matplotlib figures, exports known in-memory DataFrames to CSV, writes machine-readable manifests, writes a human-readable README and Markdown report, checks required artifacts, constructs a ZIP archive, and then validates the archive again.

The ZIP validation includes CRC testing, duplicate-name detection, and a set-equality check between the intended file set and actual archive members. Pilot checkpoints are excluded from the package by default because the code treats them as intermediate debugging artifacts rather than headline scientific outputs.

A notable design choice is that the packaging function is not executed automatically at definition time. The final notebook cell is the explicit export/download center. This reduces the chance that the archive is generated before all diagnostics and final tables exist.

18 Computational Considerations

Several parts of the implementation are motivated by Colab-scale constraints.

The dataset metadata pass is vectorized and does not decode all 17,509 images. Image paths are indexed once, pandas transformations are vectorized, and file sizes are read from filesystem metadata rather than image contents. This reduces preprocessing overhead substantially relative to per-image verification.

The archive validation result is memoized. DataLoader workers are persistent across the epochs of one experiment but explicitly shut down afterward. Completed models are moved to CPU between evaluation stages. Test inference uses `torch.inference_mode()` and device-side accumulation. These measures reduce memory pressure and avoid avoidable synchronization or worker startup costs.

The computational complexity of a single training epoch is dominated by the forward and backward passes of the selected neural network. At the data-structure level, metadata indexing and class-statistic operations are essentially linear in the number of metadata rows, while the archive integrity test scales with the amount of archive content that must be inspected. The source does not provide a closed-form runtime model, so this report does not assign fabricated timing estimates.

The training configuration also creates a clear computational trade-off: the pilot search is deliberately shallow, while final training uses the full available training split. This concentrates compute on promising configurations without turning the experiment into a large hyperparameter sweep.

19 Limitations and Technical Considerations

Several limitations can be stated directly from the source.

First, the default protocol is a single selected five-fold-derived partition rather than a full five-fold retraining study. Therefore, the existence of a `StratifiedGroupKFold` construction should not be interpreted as five independent model evaluations. The source itself describes the run as a practical 60/20/20 reconstruction.

Second, the strongest grouping key is derived from the filename convention as capture date plus instrument identifier. The implementation is appropriately cautious about calling a fallback `source_image_id` grouping a biological plant or box identifier; the code itself describes that fallback as a singleton-group leakage guard rather than a biological grouping variable.

Third, the metadata fast path does not fully decode and verify every image. Archive-level CRC validation protects the downloaded ZIP, but it is not identical to opening and decoding every individual image after extraction. The source makes this distinction explicit through `status="not_scanned"` and by leaving width and height unavailable.

Fourth, strict bitwise determinism is not enabled by default. The experiment is seed-controlled and deterministic at the design level, but backend kernel choices can still vary across hardware and software environments. The saved environment manifest is therefore important for interpreting reruns.

Fifth, model selection relies on validation top-1 accuracy alone. Macro F1 and weighted F1 are reported for the final test pass, but they are not used to choose the configuration or checkpoint. This is a coherent protocol, but it means the selection objective is narrower than the evaluation dashboard.

Finally, the source includes plotting and image-montage functionality that is useful for human inspection, but those artifacts are not evidence of biological causation. The implementation correctly limits its formal interpretation to observed classification behavior.

20 Overall Technical Assessment

The supplied implementation is technically coherent as a small research-computing system. Its strongest feature is not any one neural architecture, but the explicit connection between data integrity, split design, model selection, training, evaluation, reproducibility, and artifact auditing.

The separation between pilot and final training is clean. The test set remains isolated from configuration choice, class weights are computed from training data only, and the final model is restored to its best validation checkpoint before held-out evaluation. The grouping strategy is conservative, and the code refuses to continue when it cannot satisfy its class-completeness and group-disjointness requirements.

The software engineering is also stronger than a typical ad hoc notebook. There are small focused helpers for acquisition, path indexing, splitting, transformation, model construction, optimizer assembly, metric calculation, checkpointing, and packaging. The code uses explicit failure modes rather than silently continuing through missing labels, unresolved paths, invalid archives, or invalid splits.

The main methodological constraint is scope rather than internal incoherence: the project is a practical single-split Colab experiment, not a full statistical study across repeated seeds or repeated folds. That limitation is openly documented and can be addressed by extending the same experiment harness rather than redesigning the entire pipeline.

For an academic software portfolio, the implementation demonstrates several competencies that are directly visible in the source: translating a scientific protocol into executable controls, managing leakage risks, aligning pretrained-model preprocessing with architecture requirements, implementing controlled fine-tuning, designing reproducible data artifacts, and validating nontrivial experiment invariants with executable checks.

21 Conclusion

The project implements a complete nine-class DeepWeeds image-classification pipeline around three pretrained architectures: EfficientNet-B0, MobileNetV3-Large, and ViT-Tiny. Its computational path begins with validated dataset acquisition and official label resolution, proceeds through conservative grouped splitting and architecture-specific preprocessing, and culminates in two-stage transfer learning with validation-based checkpoint selection and held-out test inference.

From a research-engineering perspective, the most significant contribution of the code is the surrounding experimental discipline. The implementation separates training, validation, and test roles; derives class weights only from training data; records exact split metadata and checksums; exposes runtime and environment information; controls accelerator memory; and performs fourteen executable quality-control passes before packaging artifacts.

The supplied source does not support claims about which architecture is empirically best, because the numerical outcomes depend on actually running the pipeline on the real dataset. What can be concluded from the code itself is that the project is organized as an auditable, leakage-aware, reproducible classification experiment with explicit computational trade-offs suitable for a constrained research environment.

Appendix A. Code-to-Report Traceability

The following mapping gives a direct conceptual bridge from report topics to named source components. It is intended to help a reader locate the implementation without requiring the report to reproduce the full source code.

Report topic	Principal source components
Environment and reproducibility	<code>seed_everything</code> , global configuration constants, AMP setup
Dataset acquisition	<code>_mount_google_drive</code> , <code>_validate_zip_archive</code> , <code>_download_official_google_drive</code> , <code>_download_kaggle_mirror</code> , <code>_download_zenodo_verified</code> , <code>prepare_dataset_root</code>
Label resolution	<code>normalize_name</code> , <code>_normalize_species_series</code> , metadata construction, numeric-label consistency checks
Filename grouping	<code>_capture_group_from_filename</code> , <code>_derive_capture_timestamp</code> , <code>_candidate_groupings</code>
Grouped splitting	<code>_groups_have_enough_class_support</code> , <code>_try_stratified_group_folds</code> , <code>make_grouped_split</code>
Transforms and dataset	<code>build_model_transforms</code> , <code>OPPDPlantDataset</code> , <code>stratified_cap</code>
Model construction	<code>build_model</code> , <code>freeze_for_stage</code> , <code>count_parameters</code>
Optimization	<code>build_optimizer</code> , <code>run_epoch</code> , CosineAnnealingLR instantiation in <code>train_experiment</code>
Pilot search	pilot loop, <code>select_best_config</code>
Final training	<code>train_experiment</code> , final-model loop
Evaluation	<code>evaluate_predictions</code> , <code>macro_metrics</code> , prediction CSV generation, confusion matrices
Reproducibility artifacts	<code>split/config</code> manifests, SHA-256 checksum, <code>experiment_manifest.json</code> , <code>environment.json</code>
Quality control	<code>_qc_pass</code> and fourteen named validation passes
Packaging	<code>_known_result_dataframes</code> , <code>package_final_results</code>

Appendix B. Default Experimental Configuration

The supplied default configuration can be summarized as follows.

Setting	Default
Seed	42
Image size	224 × 224
Batch size	64
Compute profile	<code>colab_balanced</code>
Pilot train ceiling	6,000
Final train ceiling	120,000 (non-binding for 17,509 images)
Pilot epochs	2
Final head epochs	1
Final fine-tune epochs	6
Early-stopping patience	2 fine-tuning epochs
Minimum validation improvement	10^{-4}
Strict determinism	False
Architectures	EfficientNet-B0, MobileNetV3-Large, ViT-Tiny
Selection metric	Validation top-1 accuracy
Primary final metrics	Accuracy, Macro Precision, Macro Recall, Macro F1, Weighted F1

References

The supplied notebook explicitly lists the following primary sources for the dataset and project context:

1. DeepWeeds GitHub repository: <https://github.com/AlexOlsen/DeepWeeds>
2. DeepWeeds dataset DOI: <https://doi.org/10.5281/zenodo.7939060>
3. DeepWeeds paper DOI: <https://doi.org/10.1038/s41598-018-38343-3>